To begin the analysis, the dataset is first presented in tabular form. This provides a clear overview of the variables included and their respective values, serving as the foundation for subsequent exploration and visualization.

Next, we present a sample of the first five entries from the dataset. This preview offers a quick look at the structure of the data and the type of information contained within the file.

In [1]:
```
import pandas as pd
data_file = pd.read_csv("/home/kohao/Desktop/MY_FIRST_DATA_SCIENCE_PROJECT/HollywoodMovies.csv")
data_file.head()
```

Out[1]:

	Movie	LeadStudio	RottenTomatoes	AudienceScore	Story	Genre	TheatersOpenWeek	OpeningWeekend	BOAvgOpenWeekend	DomesticGross	ForeignGross	WorldGross	Budget	Profitability	OpenProfit	Year
0	Spider-Man 3	Sony	61.0	54.0	Metamorphosis	Action	4252.0	151.1	35540.0	336.53	554.34	890.87	258.0	345.30	58.57	2007
1	Shrek the Third	Paramount	42.0	57.0	Quest	Animation	4122.0	121.6	29507.0	322.72	476.24	798.96	160.0	499.35	76.00	2007
2	Transformers	Paramount	57.0	89.0	Monster Force	Action	4011.0	70.5	17577.0	319.25	390.46	709.71	150.0	473.14	47.00	2007
3	Pirates of the Caribbean: At World's End	Disney	45.0	74.0	Rescue	Action	4362.0	114.7	26302.0	309.42	654.00	963.42	300.0	321.14	38.23	2007
4	Harry Potter and the Order of the Phoenix	Warner Bros	78.0	82.0	Quest	Adventure	4285.0	77.1	17998.0	292.00	647.88	939.89	150.0	626.59	51.40	2007

The first step in exploring the dataset is to examine its overall structure. This provides a summary of the entries, the variables included, their data types, and the presence of any missing values. Such an overview is essential for understanding the dataset before moving into deeper analysis.

In [2]:
```
data_file.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 970 entries, 0 to 969
Data columns (total 16 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Movie             970 non-null    object
 1   LeadStudio        961 non-null    object
 2   RottenTomatoes    913 non-null    float64
 3   AudienceScore     907 non-null    float64
 4   Story             641 non-null    object
 5   Genre             691 non-null    object
 6   TheatersOpenWeek  949 non-null    float64
 7   OpeningWeekend    969 non-null    float64
 8   BOAvgOpenWeekend  945 non-null    float64
 9   DomesticGross     970 non-null    float64
 10  ForeignGross      876 non-null    float64
 11  WorldGross        914 non-null    float64
 12  Budget            897 non-null    float64
 13  Profitability     896 non-null    float64
 14  OpenProfit        895 non-null    float64
 15  Year              970 non-null    int64
dtypes: float64(11), int64(1), object(4)
memory usage: 121.4+ KB
```

Exploring the data

- In addition to examining the dataset's structure, we highlight the movies with the highest and lowest budgets.

In [3]:
```
print("the budget mean is :",data_file['Budget'].mean())
print("the biggest and smallest badget are:")
pd.concat([
    data_file.loc[[data_file["Budget"].idxmax()]],
    data_file.loc[[data_file["Budget"].idxmin()]],
])
```

```
the budget mean is : 56.11716833890747
the biggest and smallest badget are:
```

Out[3]:

	Movie	LeadStudio	RottenTomatoes	AudienceScore	Story	Genre	TheatersOpenWeek	OpeningWeekend	BOAvgOpenWeekend	DomesticGross	ForeignGross	WorldGross	Budget	Profitability	OpenProfit	Year
3	Pirates of the Caribbean: At World's End	Disney	45.0	74.0	Rescue	Action	4362.0	114.7	26302.0	309.42	654.00	963.42	300.0	321.14	38.23	2007
220	Soul Men	Weinstein	45.0	42.0	Comedy	Comedy	2044.0	5.4	2643.0	12.08	0.25	12.33	0.0	NaN	NaN	2008

- We also identify the movies with the highest and lowest ratings in the dataset.

In [4]:
```
print("the rating mean is :",data_file['RottenTomatoes'].mean())
print("the best and worst are:")

pd.concat([
    data_file.loc[[data_file['RottenTomatoes'].idxmax()]],
    data_file.loc[[data_file['RottenTomatoes'].idxmin()]]

])
```

```
the rating mean is : 51.707557502738226
the best and worst are:
```

Out[4]:

	Movie	LeadStudio	RottenTomatoes	AudienceScore	Story	Genre	TheatersOpenWeek	OpeningWeekend	BOAvgOpenWeekend	DomesticGross	ForeignGross	WorldGross	Budget	Profitability	OpenProfit	Year
499	Toy Story 3	Disney	99.0	91.0	Rescue	Animation	4028.0	110.3	27385.0	415.00	648.16	1063.16	200.0	531.58	55.15	2010
188	One Missed Call	Warner Bros	0.0	36.0	Monster Force	Horror	2240.0	12.5	5585.0	26.89	18.96	45.85	20.0	229.24	62.50	2008

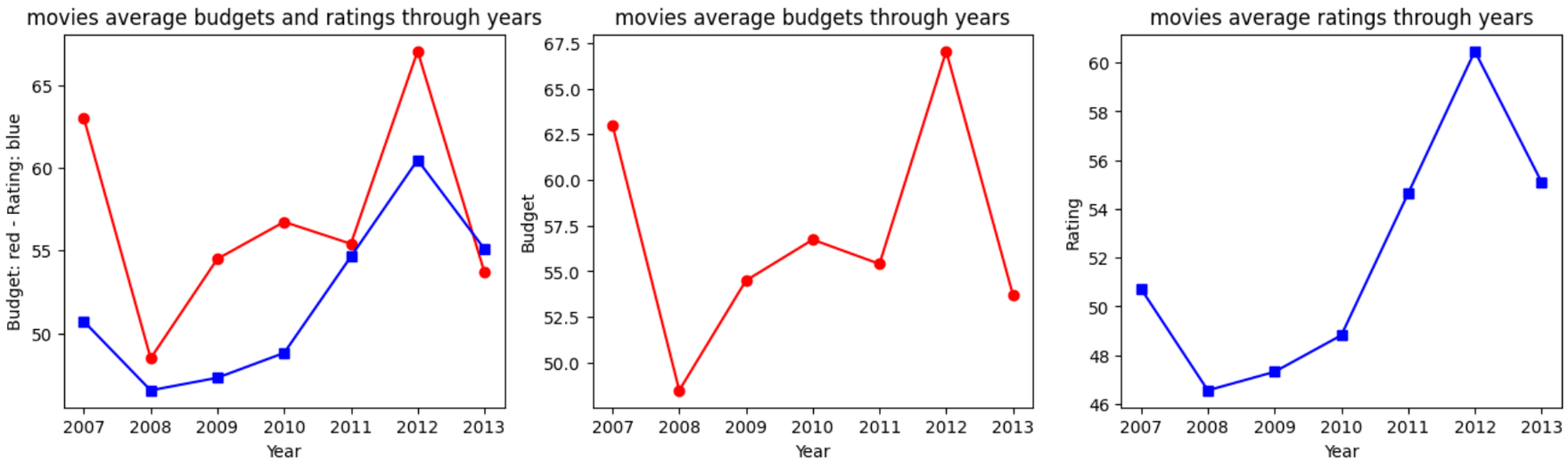
Visualization

- To better understand the dataset, we visualize the distribution of movie budgets and ratings across the years. This graphical representation highlights how production costs and audience reception have evolved over time, making it easier to identify trends, fluctuations, and potential correlations between the two variables.

In [5]:
```
import matplotlib.pyplot as plt
fig,ax = plt.subplots(1,3,figsize=(16,4))
avg_budget_per_year = data_file.groupby("Year")["Budget"].mean()
avg_rat_per_year = data_file.groupby("Year")["RottenTomatoes"].mean()

ax[0].plot(avg_budget_per_year.index, avg_budget_per_year.values,marker ="o",color = "red")
ax[0].plot(avg_rat_per_year.index,avg_rat_per_year.values,marker="s",color = "blue")
ax[0].set_xlabel("Year")
ax[0].set_ylabel("Budget: red - Rating: blue")
ax[0].set_title("movies average budgets and ratings through years")
```

```
ax[1].plot(avg_budget_per_year.index,avg_budget_per_year.values,marker="o",color = "red")
ax[1].set_xlabel("Year")
ax[1].set_ylabel("Budget")
ax[1].set_title("movies average budgets through years")
ax[2].plot(avg_rat_per_year.index,avg_rat_per_year.values,marker="s",color = "blue")
ax[2].set_xlabel("Year")
ax[2].set_ylabel("Rating")
ax[2].set_title("movies average ratings through years")
plt.show()
```



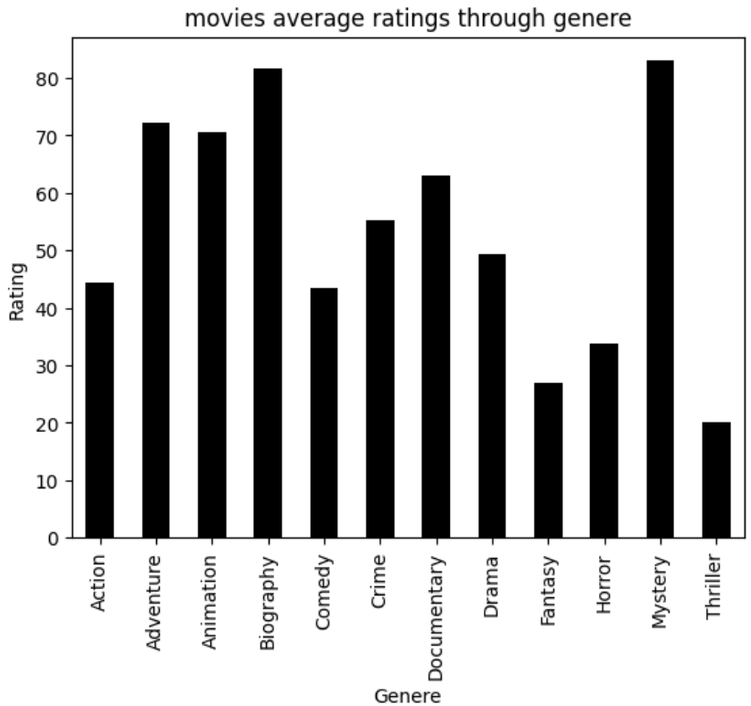
From the previous figures, we observe that movie budgets were significantly high in the early 2000s, dropped sharply around 2008, and then reached their peak in 2012. Interestingly, the rise in budgets appears to coincide with an increase in average ratings, suggesting that higher spending was often associated with better-received films.

- We visualize the distribution of movie ratings across different genres for the year 2010. This snapshot highlights how audience reception varied among genres during that specific year, offering insights into which categories achieved higher ratings and which faced more critical challenges.

In [6]:
```
movie_2010 = data_file[data_file['Year']==2010]
avg_rat_per_genere = movie_2010.groupby("Genre")["RottenTomatoes"].mean()

avg_rat_per_genere.plot(kind="bar",color = "black")
plt.xlabel("Genere")
plt.ylabel("Rating")
plt.title("movies average ratings through genere")
plt.show
```

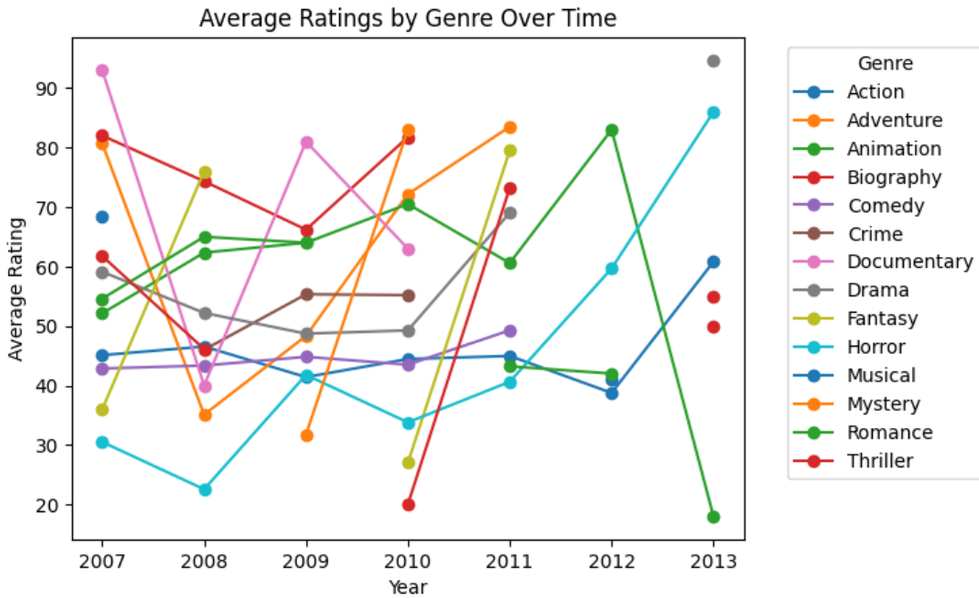
Out[6]:
```
<function matplotlib.pyplot.show(close=None, block=None)>
```



From the 2010 genre ratings figure, we can see that Biography and Mystery films achieved the highest average ratings compared to other genres. Following them, Adventure and Animation movies also performed strongly, though with slightly lower averages.

- After examining ratings by genre for the year 2010, we now extend the analysis to cover the entire dataset. This visualization illustrates how average ratings evolve across genres over multiple years, revealing long-term trends, fluctuations, and shifts in audience preferences. By comparing genres over time, we gain a broader perspective on which categories maintain consistent reception and which experience notable changes.

In [7]:
```
avg_rat_per_year_genre = data_file.groupby(["Year","Genre"])["RottenTomatoes"].mean().unstack()
avg_rat_per_year_genre.plot(kind="line", marker="o")
plt.xlabel("Year")
plt.ylabel("Average Rating")
plt.title("Average Ratings by Genre Over Time")
plt.legend(title="Genre", bbox_to_anchor=(1.05, 1), loc="upper left")
plt.show()
```



From this figure, we observe how movie genre ratings shifted over the years. By 2013, audience interest in romance films had noticeably declined, while ratings for horror movies showed the opposite trend, reflecting a growing appeal in that genre.

Finally, we extend the analysis to include predictions of average ratings for upcoming movies. This forecast is based on historical trends observed in the dataset and provides an estimate of how audience reception may evolve in future releases. While the prediction offers a mathematical perspective on potential rating outcomes, it should be interpreted with caution, as real-world factors such as genre shifts, production quality, and audience preferences can significantly influence actual results.

In [8]:
```
avg_rat_th_year = data_file.groupby("Year")["RottenTomatoes"].mean()
years = avg_rat_th_year.index.values.reshape(-1,1)
rating = avg_rat_th_year.values

from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

poly = PolynomialFeatures(degree=2)
x_poly = poly.fit_transform(years)
model = LinearRegression()
model.fit(x_poly,rating)
```

Out[8]:

LinearRegression

▸ Parameters

fit_intercept	True
copy_X	True
tol	1e-06
n_jobs	None
positive	False

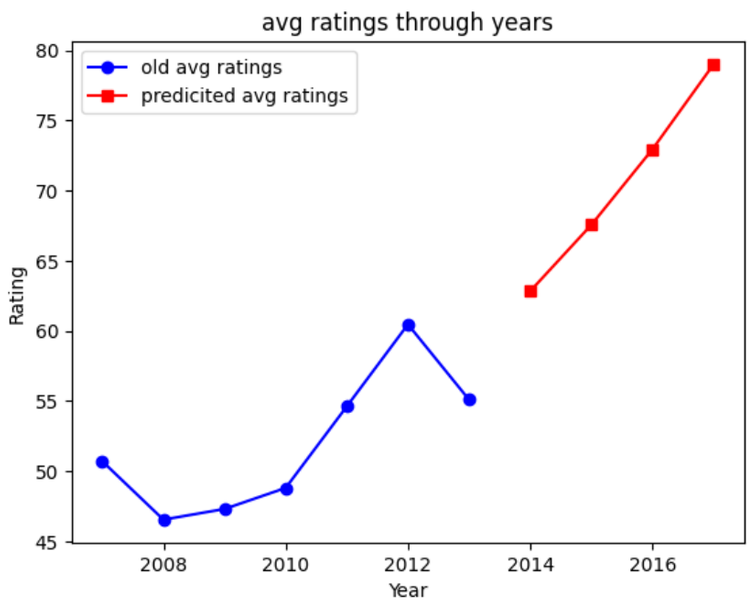
In [9]:
```
import numpy as np
fu_years = np.array([2014,2015,2016,2017]).reshape(-1,1)
fu_poly = poly.fit_transform(fu_years)
pred = model.predict(fu_poly)
print("the predicted values of the avg of the ratings of the movies in the future is :",pred)
```

```
the predicted values of the avg of the ratings of the movies in the future is : [62.84084944 67.56440164 72.95425977 79.01042383]
```

We visualize the historical average ratings across the years alongside the predicted values. This combined view allows us to compare past audience reception with the forecasted trend, highlighting how the model projects ratings to evolve. By placing actual averages and predicted averages together, the visualization provides a clearer perspective on the reliability and direction of the prediction.

In [10]:
```
plt.plot(avg_rat_per_year.index,avg_rat_per_year.values,marker = "o",color = "blue",label = "old avg ratings")
plt.plot(fu_years,pred,marker="s",color= "red",label = "predicited avg ratings")
plt.title("avg ratings through years")
plt.xlabel("Year")
plt.ylabel("Rating")
plt.legend()
```

Out[10]:
```
<matplotlib.legend.Legend at 0x7f58f9641450>
```



This prediction is purely mathematical: based on past years, the model suggests that the average movie rating will continue to rise, reaching around 80 by 2017. It reflects a simple linear regression forecast, which assumes a steady upward trend.

Research conducted by: Mouhammed Nour Qara Hammoud

Dataset source: Reisanar on GitHub